Machine learning assisted network anomaly detection

Final capstone presentation

Christopher Hazell
October 26 2026

Project overview

A working local comparison system

Threshold baseline + Random Forest

Shared metrics, error examples and exportable evidence

Strong internal scores did not establish reliable transfer

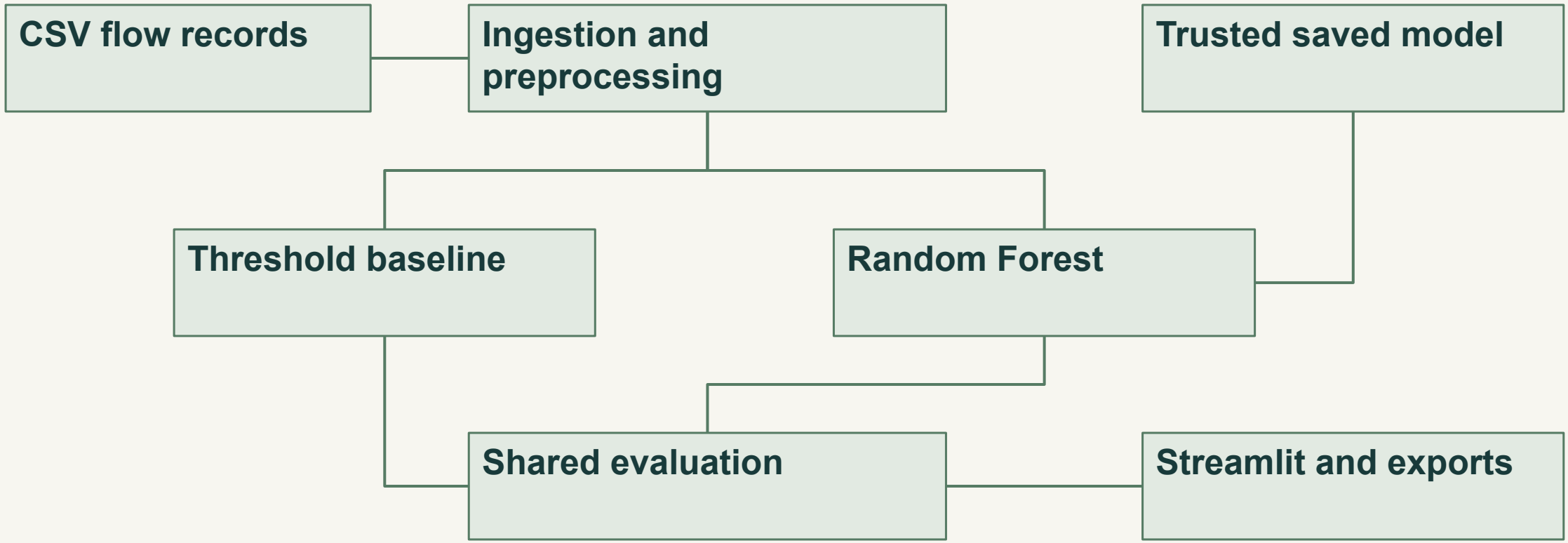
Implemented and evaluated as an academic batch-analysis prototype.

Problem objectives and scope

Objective	Implementation
Compare detection methods	One dataset preparation and evaluation path
Expose useful evidence	Metrics, confusion matrices and error examples
Verify the integrated system	Regression, workload and recovery exercises

Scope: stored CICIDS2017 flows. Live capture, blocking, authentication and durable audit storage are future work.

Implemented system architecture



Direct Python calls in one application; no database or REST API in the implemented path.

Core detection logic

Threshold baseline

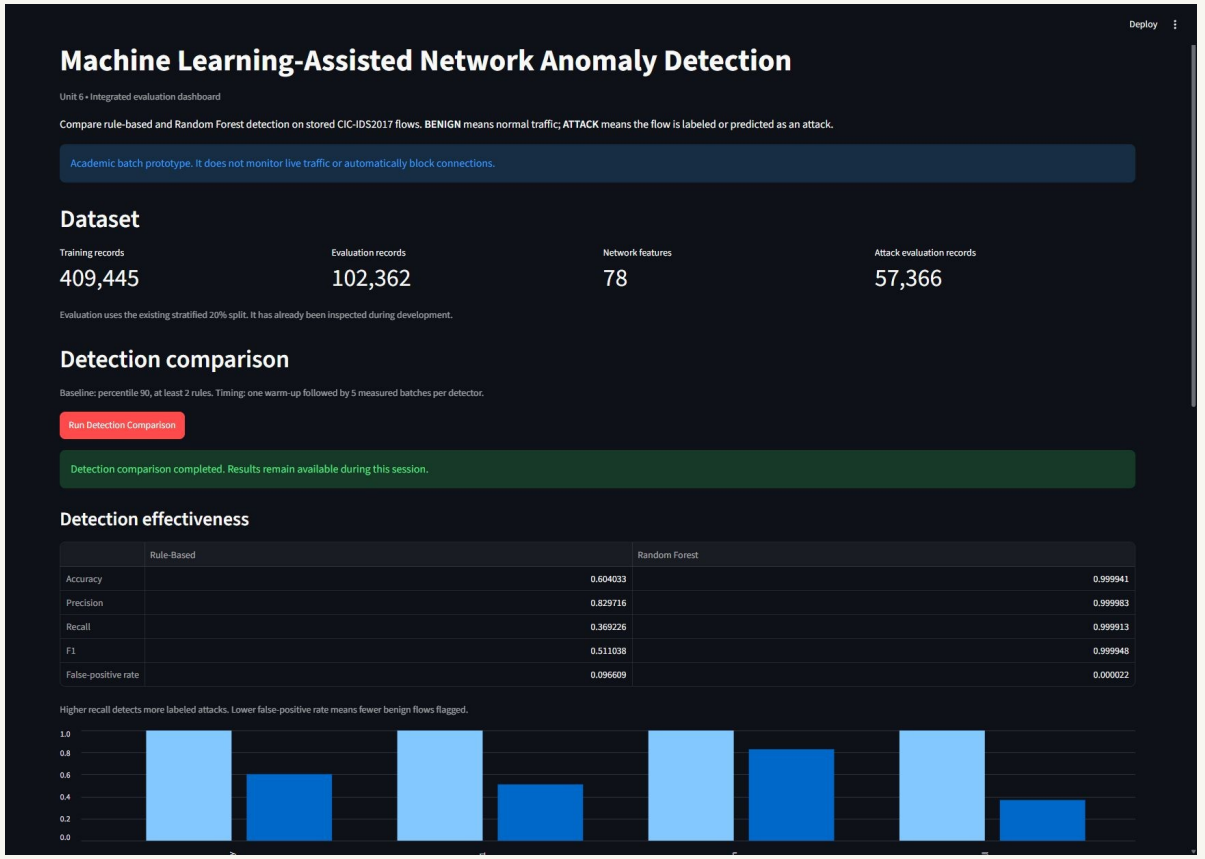
```
rule_hits = (values > thresholds).sum(axis=1)
return (rule_hits >= self.minimum_rules).astype(int)
```

90th percentile of benign training rows • 5 features • minimum 2 exceedances

```
RandomForestClassifier(n_estimators=100, random_state=42,
    class_weight="balanced_subsample", n_jobs=-1)
```

Actual output example 294575: true BENIGN → baseline ATTACK / forest BENIGN. Forest input: 78 features, excluding labels.

Integrated application output



Historical Unit 7 screenshot. The live demo uses the same inspected-split workflow.

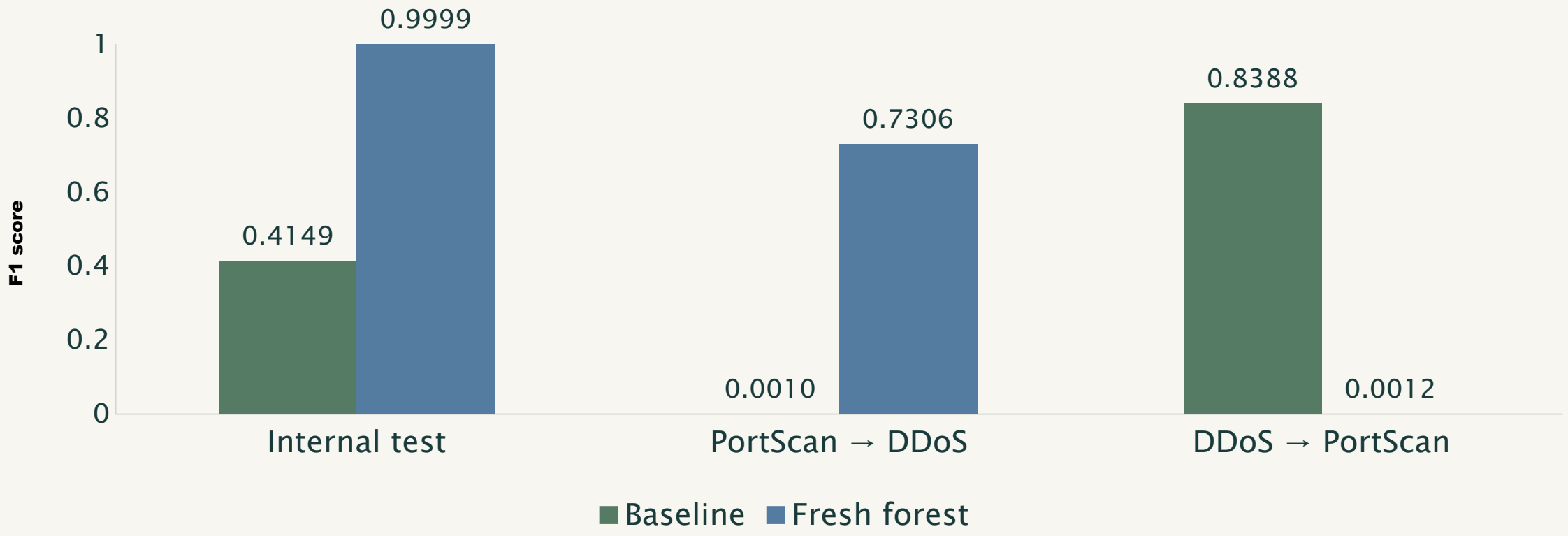
- Run comparison
- Inspect errors
- Export CSV and JSON
- Clear session results

Historical inspected split results

Historical evaluation	Baseline	Saved forest
F1 score	0.511038	0.999948
False positives	4,347	1
Missed attacks	36,185	5

102,362 evaluation rows. Audit found 21,080 test rows with a feature vector also in training. These scores require caution.

Fresh models show limited transfer



Fresh models; zero exact-vector overlap across partitions. Same previously used corpus, not independent external validation.

Security checks and reliability

72 passing regression tests

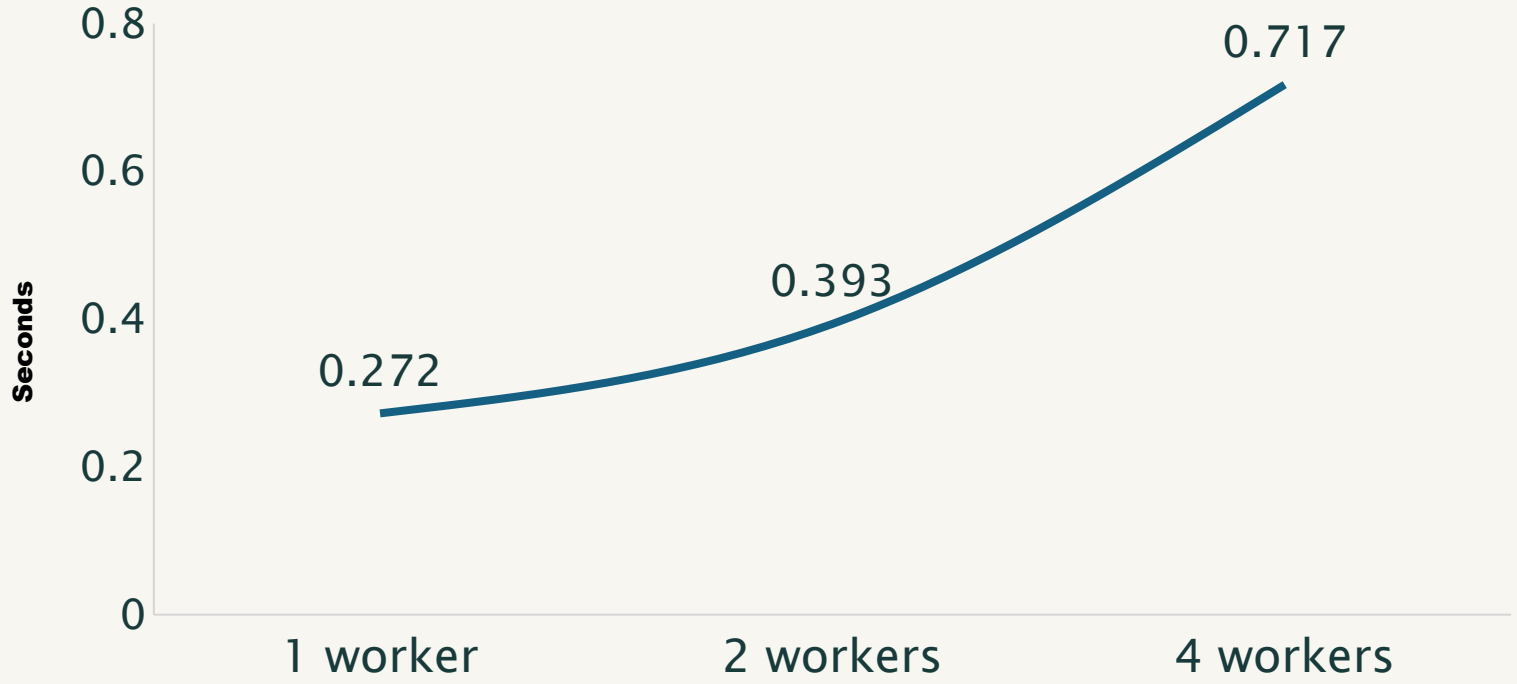
Fixed unsupported labels and duplicate normalized headers

Updated Certifi; verified restricted non-root Docker execution

Load model files only from trusted sources

Unresolved: 37 Pillow advisory records; reported fix conflicts with the current Streamlit dependency requirement.

Backend workload observations



108 operations
3 batch sizes
Consistent confusion counts

Largest batch shown
102,362 rows

Short warm backend tests; queue wait excluded. Background activity uncontrolled. These are not browser capacity measurements.

Executed backup and recovery

Recovery check	Observed result
Source, data and model integrity	91 restored files matched SHA-256 hashes
Restore through verified comparison	6.48 seconds on this workstation
Altered model copy	Integrity failure detected before loading

Local restore using the existing Python environment. Fresh-machine, offsite and disaster recovery remain untested.

Conclusions and future work

Delivered an inspectable comparison workflow

Evaluation design changed the interpretation of results

Next: independent data, transfer diagnosis and dependency remediation

Then: user studies, longer browser workloads and deployment controls

Selected public artifacts: https://chrishazell13.github.io/capstone-portfolio/

Running application demonstration

Local Docker application

1. Show environment and source
2. Run comparison and interpret results
3. Export evidence and clear session state
4. Show separate Unit 8 validation evidence

Switch to the application now. The dashboard uses the historical inspected split. Return to the references slide to close.

References

Canadian Institute for Cybersecurity. (n.d.). CICIDS2017. https://www.unb.ca/cic/datasets/ids-2017.html

Scikit-learn developers. (n.d.). Common pitfalls; model persistence; RandomForestClassifier. https://scikit-learn.org/stable/common_pitfalls.html https://scikit-learn.org/stable/model_persistence.html

Docker. (n.d.). Running containers. https://docs.docker.com/engine/containers/run/

Python Packaging Authority. (n.d.). pip-audit. https://pypi.org/project/pip-audit/

Hazell, C. (2026). Capstone final report and Unit 8 evidence. https://chrishazell13.github.io/capstone-portfolio/